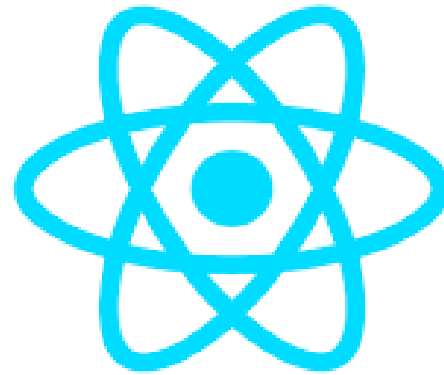




VIT[®]
AP



React JS

WEB TECHNOLOGIES

CSE-4004

MODULE 5: REACT JS

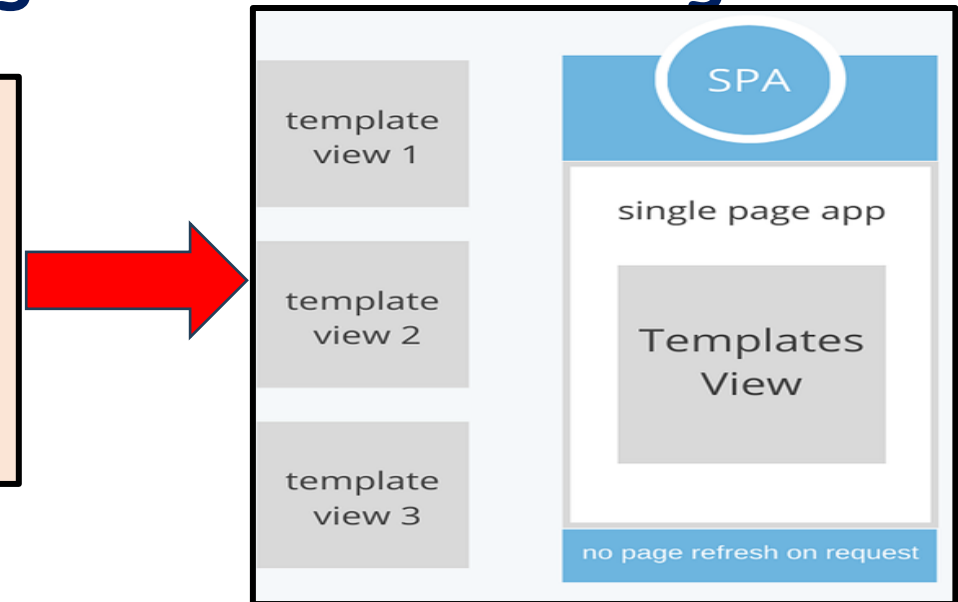
By: Dr.MD.Sirajuddin

SCOPE

WHAT IS REACT JS?

- **REACT JS**, is a front-end technology developed by Meta (Facebook).
- It is an **Open Source JavaScript Library** used to build fast and interactive User Interfaces.
- It is used to develop **Single Page Application (SPA)**
- Dynamically updates part of the webpage without reloading it.

A SPA is a type of web application that loads a single HTML page and then dynamically updates the content as the user interacts with it — without refreshing the entire page.



CORE FEATURES OF REACT

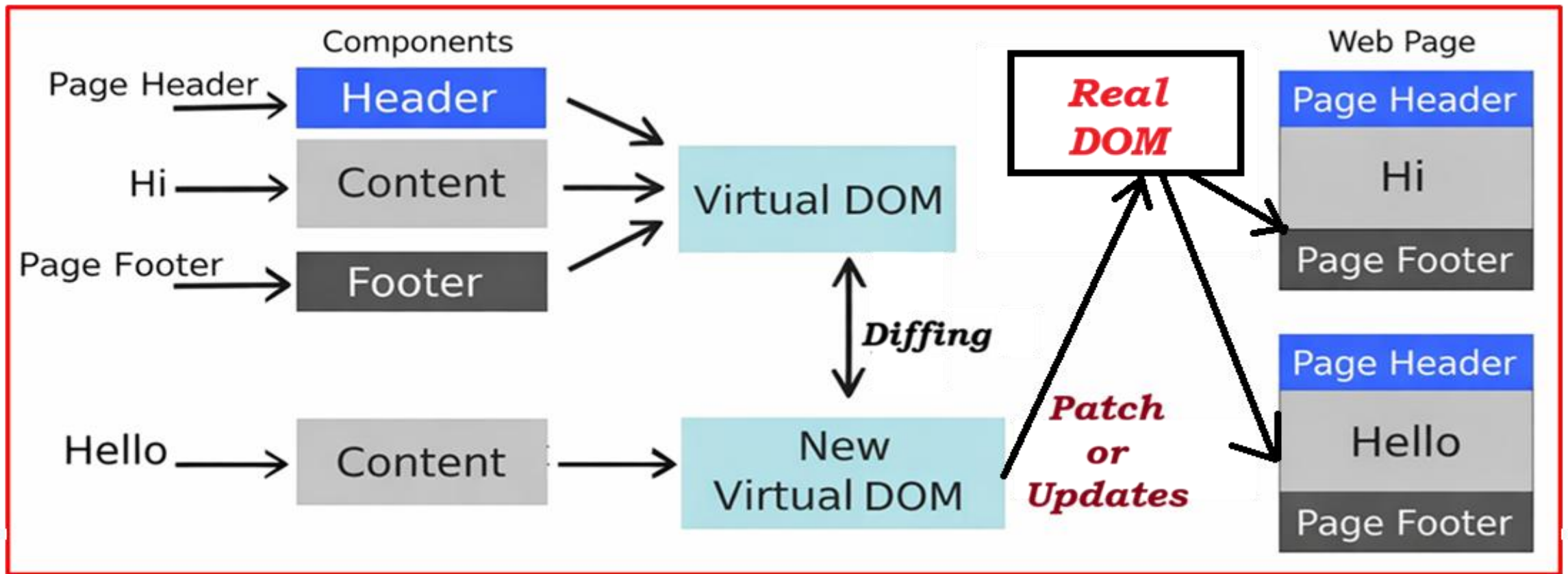
Features	Description
Component-Based	• Fundamental building block of React • It is a function or class used to build UI.
Virtual DOM	A lightweight copy of the real DOM that React uses to detect and apply changes efficiently.
One-Way Data Binding	unidirectional data flow Parent Component → Child Component
JSX	writing HTML inside JavaScript → Follows XML Rules <code>const element = <h1>Hello React</h1>;</code>
Reusable Components	Components can be reused across different parts of the app use statement in code <code>export default App;</code>
Modular Structure	Modularization of code (<i>large system → components</i>)
Hooks	Hooks are functions required to use React features (like state and lifecycle) inside function components.

SCOPE



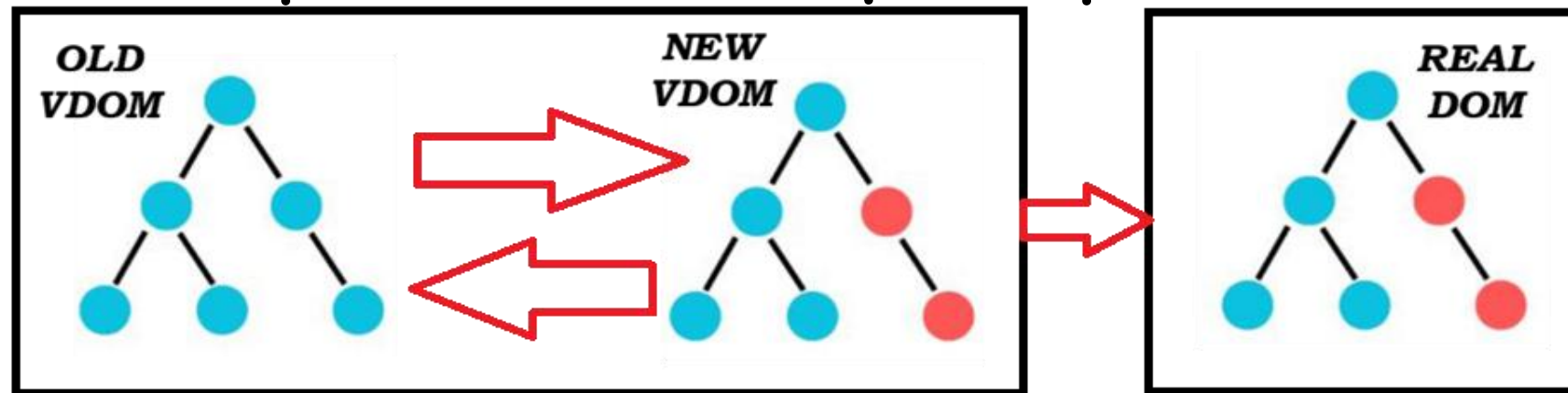
Virtual DOM

- Lightweight copy of Real DOM.
- React uses it to **update the UI** efficiently by minimizing direct manipulation of the actual DOM. (**speeds up rendering**)



Virtual DOM → Live Reload

- **Diffing & Reconciliation**: New DOM and Old DOM are compared to identify the differences, and a **patch** is prepared.
- The **patch** is applied to the real DOM.
- After each update, React throws away the **old Virtual DOM**.
- The **new Virtual DOM** becomes the baseline for the next comparison.
- Updates only the part of the webpage without reloading it.
- Works well for complex UIs with frequent updates.



TOOLS REQUIRED → REACT JS

- 1. Node JS with npm (Node Package Manager)**
- 2. VS Code Editor**
- 3. Create React App → Tool used to create React Projects**
 - *npx create-react-app myapp***
- 4. Web Browser**

REACT JS INSTALLATION

1. Install Node JS → <https://nodejs.org/en/download>

➤ LTS Version + npm (node package manager)

2. Open Command prompt

➤ C:\Users\Siraj> **node -v**

v24.14.0

➤ C:\Users\Siraj> **npm -v**

11.9.0

3. cd D:

>npx create-react-app studentapp

>cd studentapp

>npm start

E:\>npx **create-react-app** firstapp

REACT JS INSTALLATION

1. *E:\>cd firstapp*
2. *E:\firstapp>npm start*

firstapp

node_modules

public

src

.gitignore

package

package-lock

README

```
E:\firstapp>npm start
```

```
> firstapp@0.1.0 start
```

```
> react-scripts start
```

```
(node:26248) [DEP0176] DeprecationWarning: fs.F_OK is deprecated, use fs.constants.F_OK instead  
(Use 'node --trace-deprecation ...' to show where the warning was created)
```

```
(node:26248) [DEP_WEBPACK_DEV_SERVER_ON_AFTER_SETUP_MIDDLEWARE] DeprecationWarning: 'onAfterSetupMiddleware' option is deprecated. Please use the 'setupMiddlewares' option.
```

```
(node:26248) [DEP_WEBPACK_DEV_SERVER_ON_BEFORE_SETUP_MIDDLEWARE] DeprecationWarning: 'onBeforeSetupMiddleware' option is deprecated. Please use the 'setupMiddlewares' option.
```

```
Starting the development server...
```

```
Compiled successfully!
```

```
You can now view firstapp in the browser.
```

```
Local: http://localhost:3000
```

```
On Your Network: http://172.19.124.233:3000
```

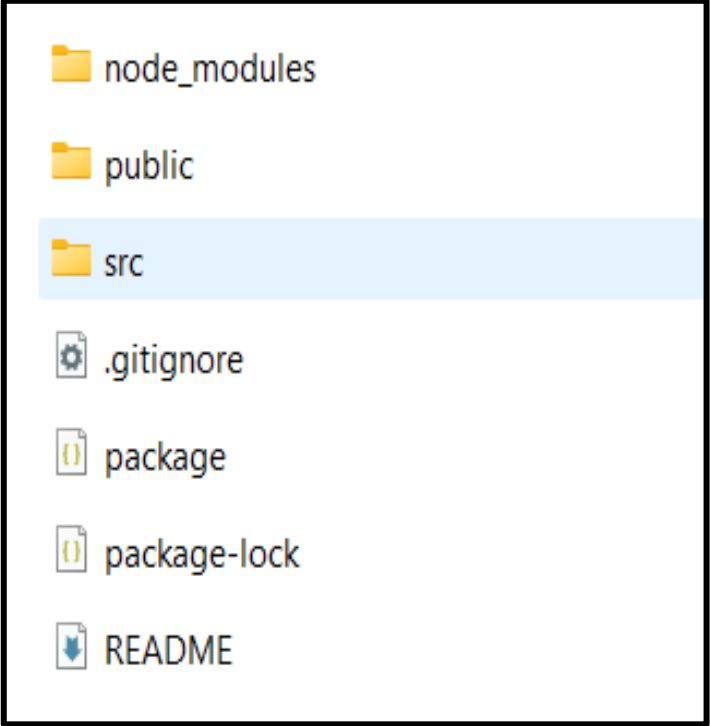
```
Note that the development build is not optimized.  
To create a production build, use npm run build.
```

```
webpack compiled successfully
```

REACT JS INSTALLATION

1. *node_modules* → modules and dependencies for react
2. *public* → Contains static files
3. *src* → important folder → react code is written in App.js
4. *package.json* → project information & dependency list

firstapp



A file explorer window showing the directory structure of a React project. The 'src' folder is highlighted in blue. The files and folders listed are: node_modules (folder), public (folder), src (folder), .gitignore (file), package (file), package-lock (file), and README (file).

- node_modules
- public
- src
- .gitignore
- package
- package-lock
- README

Write react code in App.js file in src folder

- *App.js is the root component of a React Application.*

```
function App() //component name
```

```
{
```

```
  return (
```

```
    <div>
```

```
      <h1>Hello Students</h1>
```

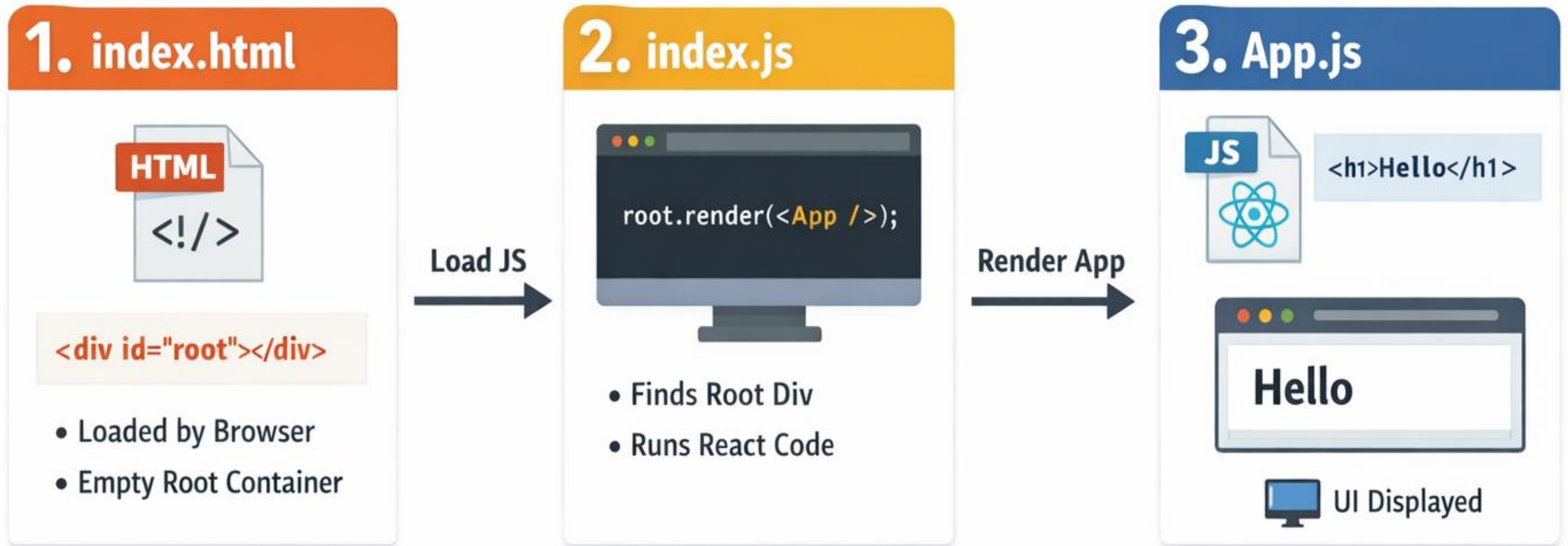
```
      <p>Welcome to React JS</p>
```

```
    </div>
```

```
  ); //returned content will appear in the browser
```

```
}
```

JSX is returned and it is displayed in browser.



Open browser → <http://localhost:3000/>

App.js is the main or root component of a React Application

```
function App() {  
  return (  
    <div>  
      <Header />  
      <Menu />  
      <Content />  
      <Footer />  
    </div>  
  );  
}
```

App.js → main component

Header, Menu, Content, Footer → child components

We can create our own components and import them into App.js.

Open browser → <http://localhost:3000/>

TEMPLATING USING JSX (JAVASCRIPT XML)

- JSX is an extension of JavaScript, primarily used with React to design User Interfaces (UI).
- Designing the UI (HTML structure) inside JavaScript using JSX
- JSX allows us to write HTML like code inside JavaScript which is then compiled into standard react element.

- A template is a layout of UI

• `function App()`

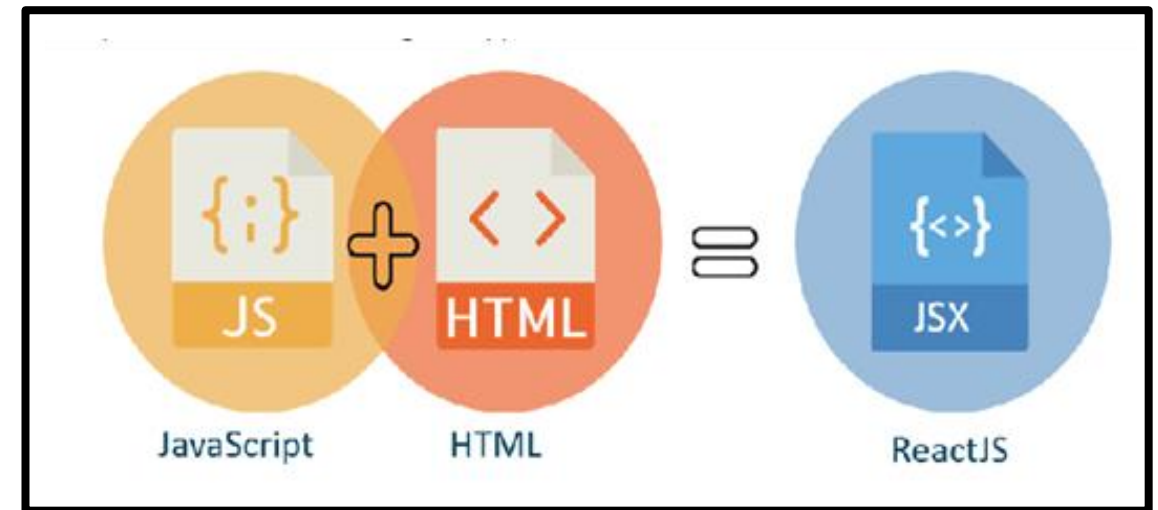
```
{  
  return <h1>Hello Students</h1>;  
}
```

- Here, `<h1>` is the template

- JSX is used to design UI

- JSX Variables - Used to display dynamic values & written inside `{ }`.

- JSX supports all JavaScript data types



JSX with Expressions

- Expressions can be specified by enclosing them in `{ }`.

Example 1: `const name = "SCOPE";`

`const element = <h1>Hello {name}</h1>;`

Example 2: `const x = 10;`

`const y = 20;`

`const element = <h2>Sum = {x + y}</h2>;`

```
function App() {  
  const name = "Students";  
  return (  
    <div>  
      <h1>Hello {name}</h1>  
      <p>Welcome to React Class</p>  
    </div> );  
}
```

JSX Execution Steps

1. Create React Component using JSX
 2. JSX is compiled into JS using Babel
 3. Creates Virtual DOM
 4. Diffing Process
 5. Update Real DOM
 6. Browser Renders Output → Final UI appear on screen
- JSX Code → Babel → JavaScript → Virtual DOM → Diffing → Real DOM → Browser Output

How JSX is Compiled

From JSX to JS & DOM

1. You write JSX

```
const element = <h1>Hello World</h1>;
```

2. Babel Compiles JSX

↓ Transpiles Code ↓

BABEL

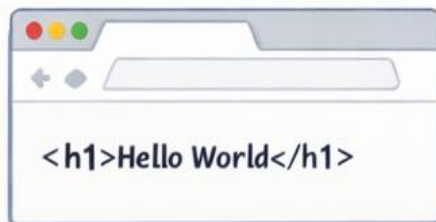
```
const element = React.createElement("h1",  
null, "Hello World");
```

3. `React.createElement()` Output:

```
{  
  type: "h1",  
  props: {  
    children: "Hello World"  
  }  
}
```

4. React Renders to the DOM

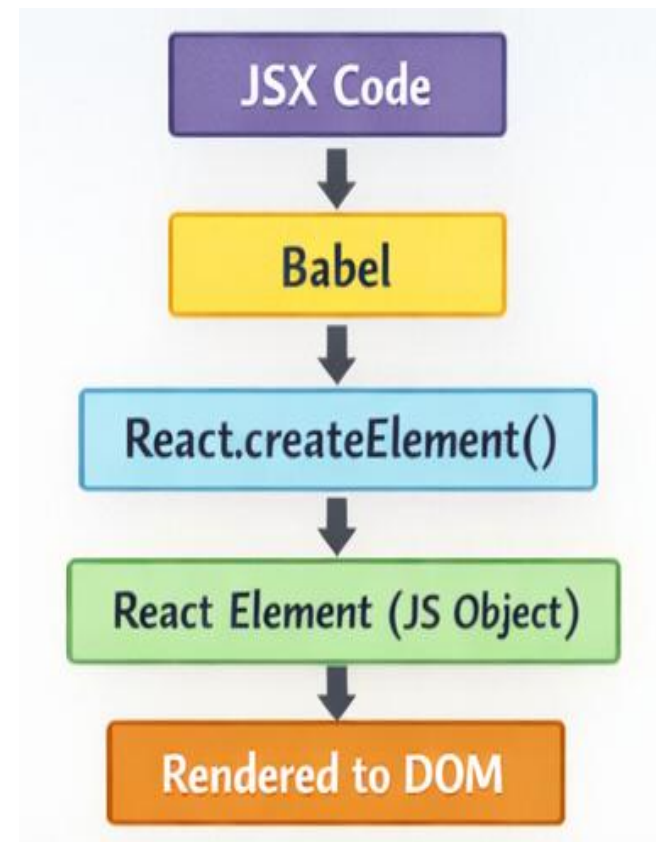
→ Converts to Real DOM



5. Final Output in Browser

```
<h1>Hello World</h1>
```

JSX to JavaScript Conversion



- JSX is not a new programming language, it's just syntax.
- `const element = <h1>Hello</h1>;`
- The tool Babel takes your JSX code and **transpiles** it.
- Browsers cannot read JSX directly, so tools like Babel convert it.
- `React.createElement(type, props, children) → React.createElement("h1", null, "Hello");`
- Javascript representation of React Element or React Element Object.

```
{  
  type: "h1",  
  props: {  
    children: "Hello World"  
  }  
}
```

- React takes this and creates a Virtual DOM object.
- Diffing
- Convert to real DOM
- React finally renders it into actual HTML in the browser.

JSX: Conditional Statements

```
function App()
{
  let isLoggedIn = true;
  if (isLoggedIn)
  {
    return <h1>Welcome User</h1>;
  }
  else
  {
    return <h1>Please Login</h1>;
  }
}
```

Ternary Operator

```
function App()
{
  let isLoggedIn = false;
  return (
    <h1>
    { isLoggedIn? "Welcome User" :
    "Please Login" }
    </h1>
  );
}
```

JSX: Loop: Map(): Iterate over array elements & render UI.

`array.map((item, index) => { return something; });`

For each element in array:

name → value

index → position (0, 1, 2)

key={index} attribute → required in JSX to access each element

function App()

{

`const students = ["Rahul", "Anita", "John"];`

`return (`

`<ul> {students.map((s, i) => ( <li key={i}> {s} </li> ))}`

`</ul>`

`);`

`}`

Component

- A Component is a reusable & independent piece of UI
- Ex: Header, Footer, Login, Button
- Each of this can be implemented as a separate component

Types of Components

1. *Functional Components: Simple JavaScript functions*
2. *Class Component: Uses ES6 classes*

1. Functional Components:

i. Create the component file

`Welcome.js`

ii. Write functional component

```
function Welcome()
```

```
{
```

```
    return <h1>Hello User!</h1>;
```

```
}
```

This is a simple JavaScript function that returns JSX.

iii. Export the Component

```
export default Welcome;
```

This allows the component to be used in other files.

iv. Import the Component in another file

```
import Welcome from './Welcome'
```

1. Functional Components:

- Components defined in the same file do not require import.
- Import is needed only when components are defined in separate files.
- *Using Component in App.js*

```
import Welcome from './Welcome';  
function App()  
{  
    return (<div> <Welcome /> </div> );  
}
```
- *<Welcome /> is not a tag, it is a custom component.*

1. Functional Components: All components in one file App.js

```
function Header() {  
  return <h1>Header</h1>;  
}
```

```
function Footer() {  
  return <h2>Footer</h2>;  
}
```

```
function App() {  
  return (  
    <div>  
      <Header />  
      <Footer />  
    </div>  
  );  
}  
  
export default App;
```

- Only one default export per file.

```
export function Header() //Inline Export
{
return <h1>Header</h1>;
}
export function Footer() {
return <h2>Footer</h2>;
}
-----
import { Header, Footer } from './File';
```

```
export { Header, Footer }; → Named Export
import { Header, Footer } from './File';
-----
export default App;
export { Header };
import App, { Header } from './File';
```

Props means Properties

- In React Props are used to pass data from one component to another.
- Usually from parent component → child component

Why Props ?

- To make components dynamic (To display dynamic output)
- To reuse components with different data

Key Points

- Props are read-only
- Passed as attributes in JSX
- Used for communication between components

Create Child Component

// Welcome.js

```
function Welcome(props)
{
  return <h1>Hello
{props.name}</h1>;
}
export default Welcome;
```

Pass data from Parent Component

import Welcome from './Welcome';

```
function App() {
  return (
    <div>
      <Welcome name="Bob" />
      <Welcome name="John" />
    </div>
  );
}

export default App;
```

Question: A company is developing an online shopping application using React. The user interface is divided into small reusable parts such as Header, ProductList, ProductItem, Cart, and Footer. Design suitable components for this application. The ProductList component passes product details to the ProductItem component. Explain how they are organized in App.js.

Solution: Organizing Components to support maintainability

```
src/  
├── App.js  
└── components/  
    ├── Header.js  
    ├── ProductList.js  
    │   └── ProductItem.js  
    ├── Cart.js  
    └── Footer.js
```

Header.js

```
function Header() {  
  return <h1>Online Shopping App</h1>;  
}  
export default Header;
```

ProductItem.js

```
function ProductItem(props) {  
  return (  
    <div>  
      <h3>{props.name}</h3>  
      <p>Price: ₹{props.price}</p>  
      <button>Add to Cart</button>  
    </div>);  
}  
export default ProductItem;
```

ProductList.js

```
import ProductItem from './ProductItem';
```

```
function ProductList() {
```

```
  return (
```

```
    <div>
```

```
      <ProductItem name="Mobile" price="15000" />
```

```
      <ProductItem name="Laptop" price="50000" />
```

```
    </div>
```

```
  );
```

```
}
```

```
export default ProductList;
```

Cart.js

```
function Cart() {  
  return <h2>Your Cart is Empty</h2>;  
}  
export default Cart;
```

Footer.js

```
function Footer() {  
  return <p>© 2026 Shopping App</p>;  
}  
export default Footer;
```

App.js

```
import Header from './Header';
import ProductList from './ProductList';
import Cart from './Cart';
import Footer from './Footer';
function App() {
  return (
    <div>
      <Header />
      <ProductList />
      <Cart />
      <Footer />
    </div>
  );
} export default App;
```

Class Component

- Create reusable UI
- Manage state and lifecycle

1. import React from "react";

2. class Hello extends React.Component → it inherits react features

```
3. render() {  
  return <h1>Hello World</h1>;  
}
```

Class component must have render() method.

4. export default Hello;

2. Class Component

```
import React from 'react';  
class App extends React.Component {  
  render()  
{  
  return <h1>Hello</h1>;  
}  
}  
export default App;
```

Class Component → Hello.js

```
import React from "react";
```

```
class Hello extends React.Component  
{  
  render() {  
    return <h1>Hello World</h1>;  
  }  
}  
export default Hello;
```

//App.js

```
import Hello from "../Hello";  
function App() {  return <Hello />; }
```

PropTypes Validation

- PropTypes are used to validate the props passed to a component.
- check whether the data coming from parent to child is correct or not.
- They are available as separate package **prop-types** in **React**
- **npm install prop-types**
- *import PropTypes from "prop-types";*
- ```
function User(props) {
 return <h1>{props.name}</h1>;
}
User.propTypes = {
 name: PropTypes.string
};
```

## Prop Types

```
function Student({ name, age, loginStatus }) {
 return (<div> <h2>{name}</h2>
 <p>{age}</p>
 <p>{loginStatus? "Yes" : "No"}</p>
 </div>
);
}
```

## //Outside the Component

```
Student.propTypes = {
 name: PropTypes.string,
 age: PropTypes.number,
 loginStatus : PropTypes.bool
};
```

**Prop Types:** String, number, bool, array, object, func, node & element

```
import PropTypes from "prop-
types";

function Box({ content }) {
 return <div>{content}</div>;
}
Box.propTypes = {
 content: PropTypes.node
};
<Box content="Hello World" />
<Box content={123} />
<Box content={<h1>Heading</h1>} />
```

```
function Title() {
 return <h1>Hello from Title
</h1>;
}

import PropTypes from "prop-
types";
function Header({ title }) {
 return <div>{title}</div>;
}
Header.propTypes = {
 title: PropTypes.element
};
<Header title={<Title />} />
```

- **Default Props:**

```
function Student({ name }) {
 return <h1>{name}</h1>;
}
```

```
Student.defaultProps = { name: "Guest"};
```

- **Required Props:**

- ```
function Student({ name }) {  
  return <h1>{name}</h1>;  
}
```

```
Student.propTypes = { name: PropTypes.string.isRequired };
```

- If Required Props missed by parent then → **Warning: Failed prop type: The prop `name` is marked as required...**

Question on props:

Create a component with name Student. The Student component should receive the following props:

- name
 - age
 - course
- i. Display the student details in a structured format.
 - ii. Create a parent component **App** and render at least 2 Student components with different data.
 - iii. Use PropTypes to validate the properties.

State:

- State is a built-in object used to store data that can change over time in a component. Accessed by using **Hook**
- **Hook** is a special function to manage state in functional components

Examples: Cart items in Shopping App, Login Status

- *These values change → so they are state*
- *To make UI dynamic*

1. *import { useState } from 'react';*

{useState} is a hook used to create state

2. *const [count, setCount] = useState(0);*

count → current value (state)

setCount → function to update value

useState(0) → returns an array , Array destructuring

State:

3. Render UI

```
return (<button> Total: {count} </button> );
```

4. Handle Click Event

```
onClick={() => setCount(count + 1)}
```

- i. When button is clicked:
- ii. `setCount()` is called
- iii. Value increases (`count + 1`)
- iv. State updates
- v. Component re-renders
- vi. UI updates automatically

```
import { useState } from 'react';
function Counter()
{
  const [count, setCount] = useState(0);
  return ( <center>
    <button
      onClick={() => setCount(count + 1)}
      style={{ fontSize: "24px", padding: "15px 30px" }}
    >
      Total: {count}
    </button> </center>
  ); }
function App()
{
  return <Counter />;
} export default App;
```

Shopping Cart Example

```
import React, { useState } from "react";
```

```
function App() {  
  const [cart, setCart] = useState([]);  
  return (  
    <div>  
      <h2>Cart Items: {cart.length}</h2>  
    </div>  
  );  
}
```

```
const addToCart = (product) => {  
  setCart([...cart, product]); // add new item  
};
```

Component Lifecycle

Component lifecycle means the different stages a component goes through from creation to removal in the DOM.

1. **Mounting Phase:** Component is created and added to the DOM
Constructor() → render() → componentDidMount()
2. **Updating Phase:** State or Prop changes
shouldComponentUpdate() → render() → componentDidUpdate()
3. **Unmounting Phase:** Component is removed from DOM
componentWillUnmount()

Mounting Phase

- **constructor(props)** *//receives props to initialize component*
- **super(props);** *//calls parent class constructor React.Component*
- allows **this.props** accessible inside the constructor
- In class props are attached to the component instance.
- So we access them using **this.props**
- **render()** to create UI and display on the screen
- **componentDidMount()** → to perform side effects after the component is added to the DOM → API Calls, Timers, Event listeners

Mounting Phase

```
class Demo extends React.Component {  
  constructor(props) { //receives props to initialize component  
    super(props); //calls parent class constructor React.Component  
    console.log("1. Constructor called");  
    this.state = { count: 0 };  
  }  
  render() {  
    console.log("2. Render method called");  
    return <h1>Hello World</h1>;  
  }  
  componentDidMount() {  
    console.log("3. Component Mounted");  
  }  
}  
export default Demo;
```

Update Phase

- `shouldComponentUpdate()` → update should happen or not using true or false
- `render()` → Updating UI
- `componentDidUpdate()` → Runs after component is updated in DOM.

Update Phase

```
shouldComponentUpdate() {  
  console.log("1. Should Component Update?");  
  return true; // allow update  
}
```

```
render() {  
  console.log("2. Render method called");  
  return ( <div> <h1>{this.state.count}</h1>  
    <button onClick={this.increase}>Click</button>  
    </div> ); }
```

```
componentDidUpdate() {  
  console.log("3. Component Updated");  
}
```

```
increase = () => { this.setState({ count: this.state.count + 1 });};  
}
```

Unmounting Phase

- This phase occurs when a component is removed from the DOM.
- **ComponentWillUnmount()** → called just before the component is destroyed or removed from DOM.
- Used for clean up operations

Unmounting Phase

```
import { useState } from "react";
import Demo from "../Demo";
function App() {
  const [show, setShow] = useState(true);
  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <h1>React Lifecycle Demo</h1>
      <button onClick={() => setShow(!show)}>
        Toggle Component
      </button>
      <hr />
      {show && <Demo />} //Conditional rendering
    </div>
  );
}
export default App;
```

useEffect

- It is hook function used in functional components
- It performs side effects like - DOM update, API Calls, Timers etc
- `useEffect(() => { // code to run });`

How to work with useEffect()

1. `useEffect(() => { console.log("Runs on every render"); });`
2. `useEffect(() => {
 console.log("Component Mounted");
}, []);`
[] → empty dependency array. Runs only once.
3. `useEffect(() => {
 console.log("Count updated");
}, [count]);`
Runs when count changes

```
import { useState, useEffect } from "react";
function Demo() {
  const [count, setCount] = useState(0);
  useEffect(() => {
    console.log("Component Mounted / Updated"); //updating
    return () => {
      console.log("Component Unmounted"); //clean-up
    };
  }, [count]);
  return (<div>
    <h2>{count}</h2>
    <button onClick={() => setCount(count + 1)}>Click</button>
  </div>
  );
}
export default Demo;
```

useEffect in Online Shopping APP

1. When the user opens a product page, the app must fetch product data from a server.

```
useEffect(() => { fetchProducts(); }, [ ]);
```

Runs only once on opening products page.

2. Updating Cart Total Price

```
useEffect(() => { calculateTotal(); }, [cart]);
```

3. Limited Time Deals (Festive Deals, Limited Offers)

```
useEffect(() => {  
  const timer = setInterval(() => { updateTime(); }, 1000);  
  return () => clearInterval(timer); }, []);
```

return remove timer when user navigates to other page or when component is removed from DOM.

```
const [timeLeft, setTimeLeft] = useState(60); // 60 seconds  
function updateTime() { setTimeLeft(prev => prev - 1); }
```

Event Handling: Used to perform actions in response to user interactions using event handler functions.

- Event → user action (Mouse events, keyboard events)
- Event handler → function that performs the action
- React uses camelCase for event names (onClick, onChange)

Example 1: Button Click Event Handling

```
function App()
{
  function handleClick()
  {
    alert("Button Clicked");
  }
  return (<button onClick={handleClick}>Click Me</button>);
}
```

Event Handling:

Example 2: Inline Event Handling

```
function App()  
{  
  return (<button onClick={() => alert("Inline Click")}> Click Me  
    </button> );  
}
```

Example 3: Passing Parameter to Event

```
function App() {  
  function greet(name) {  
    alert("Hello " + name);  
  }  
  return (<button onClick={() => greet("SCOPE")}> Greet  
    </button>);  
}
```

Event Handling:

Example 4: Passing Event Object

- React automatically passes an event object (e) to event handler function.

```
function App() {  
  function handleChange(e)  
  {  
    console.log(e.target.value);  
  }  
  return (<input type="text" onChange={handleChange} />);  
}
```

e → the event object

e.target → the HTML element where the event happened

e.target.value → current text inside the input

e.target.textContent → for button only (to access button label)

Event Handling:

Example 5: Prevent Default Behaviour

- Form refreshes the page on submit
- `e.preventDefault();`
- Without `preventDefault()` → page reloads
- With `preventDefault()` → no reload
- Prevent default behavior means stopping the automatic action associated with an event using `e.preventDefault()` method.

Event Handling:

Example 5: Prevent Default Behaviour

```
function App()
{
  function handleSubmit(e)
  {
    e.preventDefault();
    alert("Form Submitted");
  }
  return ( <form onSubmit={handleSubmit}>
    <button type="submit">Submit</button>
    </form>
  );
}
```

Event Handling:

Example 6: useState

```
import { useState } from "react";  
function Counter() {  
  const [count, setCount] = useState(0);  
  function increment()  
  {  
    setCount(count + 1);  
  }  
  return (<>  
    <h2>{count}</h2>  
    <button onClick={increment}>Increase</button>  
  </>  
  );  
}
```

Event → changes state → UI updates

Event Handling:

Example 7: Multiple Events

```
function App() {  
  function handleClick() {  
    console.log("Clicked");  
  }  
  function handleMouseOver() {  
    console.log("Mouse Over");  
  }  
  return (<button onClick={handleClick}  
    onMouseOver={handleMouseOver}>  
    Hover or Click  
  </button>  
);  
}
```

Event Handling:

Example 8: Conditional Event Handling

```
function App() {  
  function handleClick(isLoggedIn)  
{  
    if (isLoggedIn) {  
      alert("Welcome User");  
    } else {  
      alert("Please Login");  
    }  
  }  
  return (<button onClick={() => handleClick(true)}>Check  
</button>  
);  
}
```

Forms and User Input

- In React, forms are used to collect user input (like text, passwords, selections, etc.).
- React handles form data using state and event handlers.
- You add a form with React like any other element.

function MyForm()

```
{    return( <form> <label> Enter your name: <input type="text" />
          </label> </form> );
}
```

- This will work as normal, the form will submit and the page will refresh.
- But this is generally not what we want to happen in React.
- We want to prevent this default behavior and let React control the form.

HTML FORM V/S REACT FORM

- In standard HTML, form elements maintain their own value based on user input.
 - For example, an `<input type="text">` field keeps track of its own value in the HTML DOM.
 - In React, the value of the form element is kept in the component's state property and updated only with the `setState()` function.
 - React provides a way to manage form data through component state, leading to what are known as "controlled components."
 - A form element whose value is controlled by React state is called Controlled Component.
1. `import React, { useState } from "react";`
 - Importing React and the `useState` hook.
 - `useState` helps us store and manage data (state) in a component.

2. Creating functional component

- function Demo()

3. `const [name, setName] = useState(" ");`

4. `const handleChange = (event) => {
 setName(event.target.value); };`

- `handleChange` function runs when user types in the input.
- `event` is a **React synthetic event**. (Cross browser compatibility)
- It is automatically passed when the user interacts with the input
- `event.target.value` gives the current input value.
- `setName()` is a state updater function from `useState`. It updates the state with the new value.

5. `<input value={name} onChange={handleChange} />`

- `value` comes from state
- `onChange` updates state

```
import React, { useState } from "react";
function App() {
  // Step 1: Create state
  const [name, setName] = useState("");
  // Step 2: Handle input change
  const handleChange = (event) => { setName(event.target.value); };
  return ( <div>
    <h2>React Form - User Input</h2>
    <input type="text" placeholder="Enter your name"
      value={name} onChange={handleChange} />
    <p>You typed: {name}</p>
  </div> ); }
export default App;
```

React Form - User Input

You typed: SCOPE

User types → event → setState → re-render → UI updates

- When a user performs an action, the browser generates a DOM event object and automatically passes it as an argument to the event handler function.
- This object contains details about the event such as the target element, event type, and other related data.

Accessing DOM

- JavaScript Code:
- ```
document.getElementById("btn").addEventListener("click",
function(e) { console.log(e); });
```

### **HTML Element:**

- ```
<button id="btn">Click Me</button>
```

Limitations:

- i. Manually attach listeners
- ii. If 10 buttons then 10 handlers, increases memory usage.
- iii. Manual clean up.

React Event

- i. No direct DOM Access
- ii. Event delegation, one **root listener** to handle all events. A root listener is a single event listener attached at the top (root) of the app that handles events for all components.

ReactDOM.createRoot(document.getElementById("root")).render(<App />);

<div id="root"></div> root container

- iii. Automatic cleanup. React removes event handlers (unmounting)
- iv. Browser compatibility

User clicks button



DOM event created



Event bubbles to root div



React's root handler catches it



React finds your button



React calls handleClick()

```
import React, { useState } from "react";
function App() {
  const [formData, setFormData] = useState({ uname: "", email: "",
                                             password: "" });
  const handleChange = (event) => { const { name, value } = event.target;
  setFormData({ ...formData, [name]: value }); };

  const handleSubmit = (event) => { event.preventDefault();
    alert(`Username: ${formData.uname}\nEmail:
    ${formData.email}\nPassword: ${formData.password}`
    );
  };
  return ( <div>
    <h2>Registration Form</h2>
    <form onSubmit={handleSubmit}>
      <input type="text" name="uname" placeholder="Enter Username"
value={formData.uname} onChange={handleChange} /> <br /><br />
```

```
<input type="email" name="email" placeholder="Enter Email"
value={formData.email} onChange={handleChange} />
<br /><br />
```

```
<input type="password" name="password" placeholder="Enter Password"
value={formData.password} onChange={handleChange} />
<br /><br />
```

```
<button type="submit">Register</button>
```

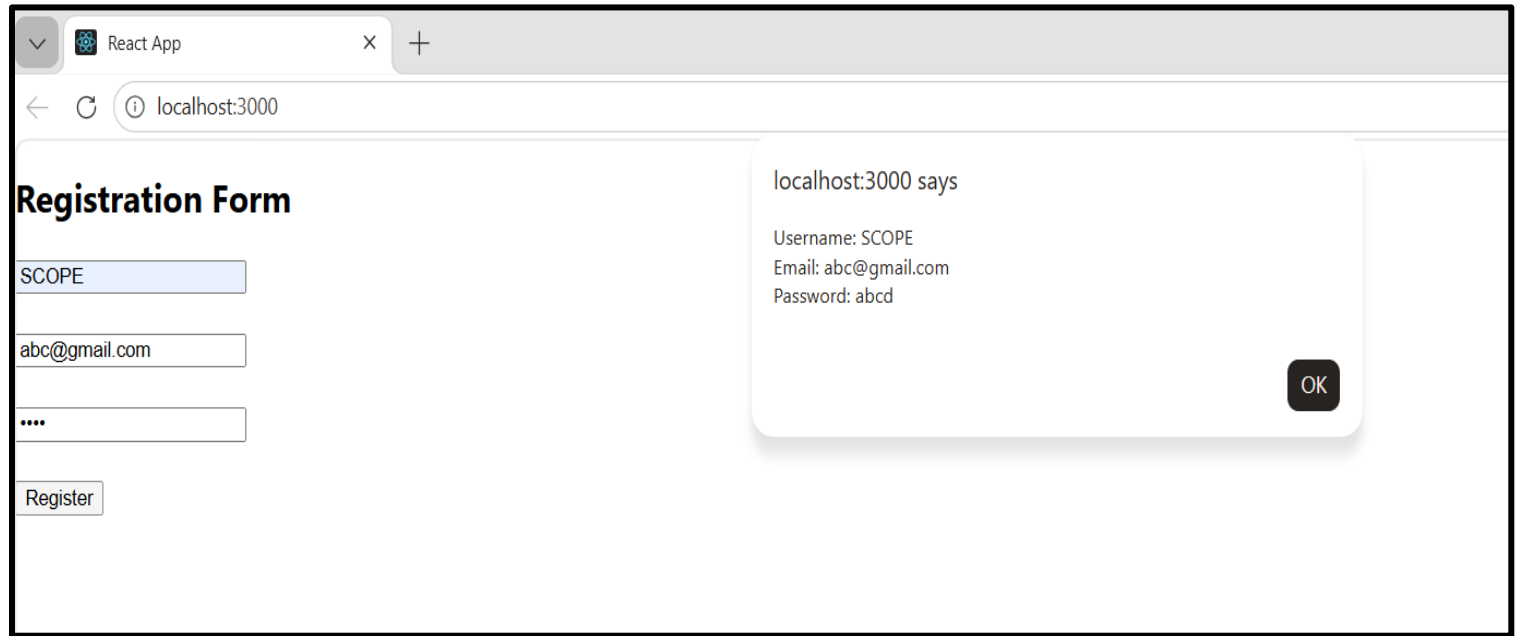
```
</form>
```

```
</div>
```

```
);
```

```
}
```

```
export default App;
```



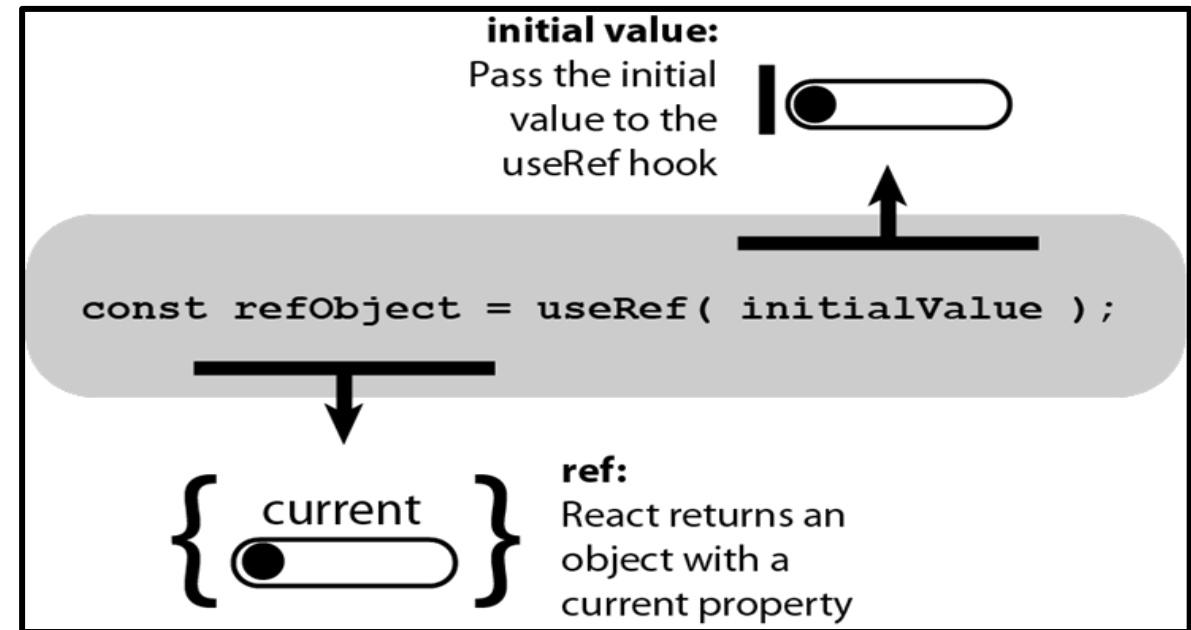
The screenshot shows a web browser window titled "React App" at the URL "localhost:3000". The page displays a "Registration Form" with three input fields: "Username" (containing "SCOPE"), "Email" (containing "abc@gmail.com"), and "Password" (containing "...."). Below the inputs is a "Register" button. A confirmation dialog box is open on the right, titled "localhost:3000 says", showing the entered details: "Username: SCOPE", "Email: abc@gmail.com", and "Password: abcd". An "OK" button is at the bottom right of the dialog.

- i. formData → holds the current state (data)
- ii. setFormData → function to update state
- iii. {
 uname: "",
 email: "",
 password: ""
} This is the initial value of state
- iv. ...formData copies existing data, and [name]: value updates the specific field in that copied data.
- v. [name] is a dynamic key. name comes from input field.
- vi. handleSubmit → function name
- vii. (event) → event object (React SyntheticEvent)
- viii. This function runs when the form is submitted
- ix. event.preventDefault(); stops the default browser behavior

useRef:

- It is a React Hook that creates a reference object used to access DOM elements.
- Used to store value related to the component.
- Used to store mutable value that persist across renders without causing re-rendering.(Update value silently without updating UI)
- To store and update data efficiently when UI update is NOT required.
- **const myRef = useRef(0);**
- Creates an object

```
{
  current: 0
}
```
- myRef is the reference object



Accessing DOM Using useRef

- `const inputRef = useRef();` //creates reference object with `{ current: undefined }`
- Attach reference object to input `<input ref={inputRef} />`
- React sets `inputRef.current = <input DOM element>`
- `inputRef` $\rightarrow$ `{ current: <input element> }`
- `alert(inputRef.current.value);` //access input directly
- In a form input, the entered value is stored in the DOM element's `.value` property, and `useRef` is used to access that element using `inputRef.current`.

useRef:

```
import { useRef } from "react";
```

```
function App()
```

```
{
```

```
  const countRef = useRef(0);
```

```
  const handleClick = () => {
```

```
    countRef.current = countRef.current + 1;
```

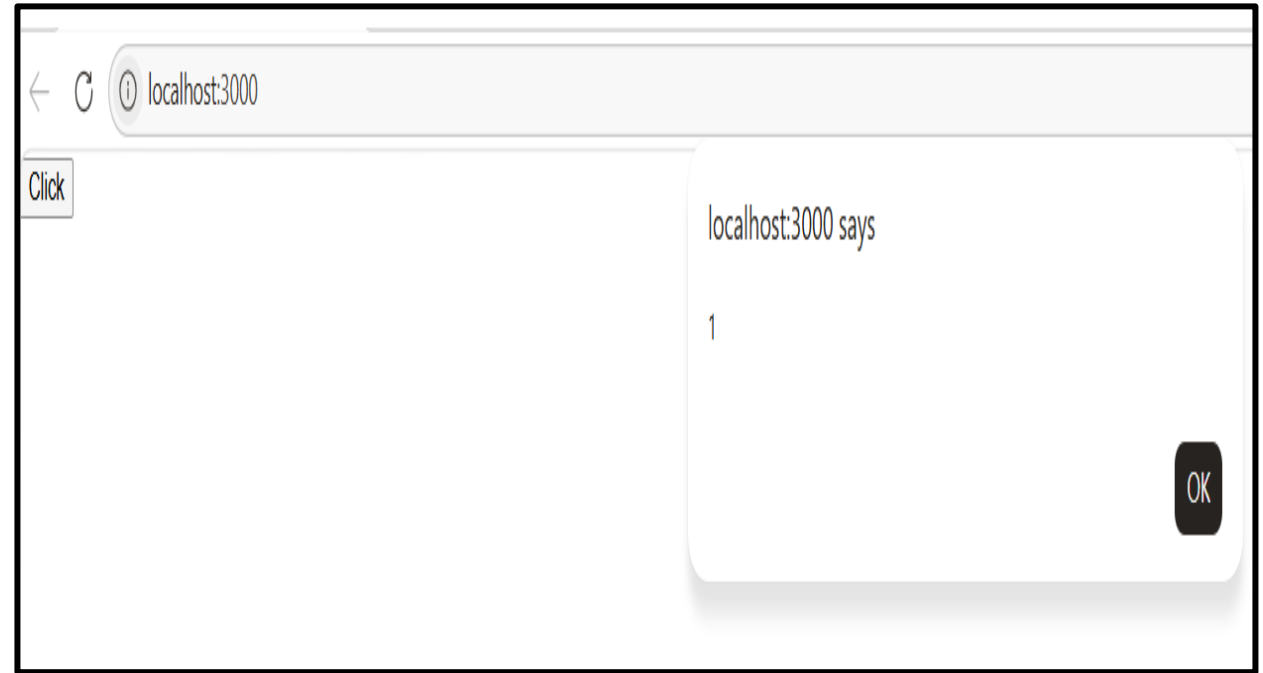
```
    alert(countRef.current);
```

```
  };
```

```
  return (<div> <button onClick={handleClick}>Click</button> </div>);
```

```
}
```

```
export default App;
```



```
import { useRef } from "react";
function App() {
  const inputRef = useRef();
  const handleClick = () => { alert(inputRef.current.value);};
  return ( <div>
    <input ref={inputRef} type="text" />
    <button onClick={handleClick}>Show Value</button>
  </div>
  );
} export default App;
```



<input ref={inputRef}> connects the input element to the ref so we can access it directly using `inputRef.current`.

React Routing

- React Routing is a technique used to display different components based on the URL in a single page application (SPA).
- It is used to handle navigation (routing).
- It is implemented using React Router.
- It performs URL → Component mapping

Why it is required?

- Without proper navigation, Clicking links reloads the whole page.
- App becomes slow

Advantages:

- With React Routing- No reload, fast navigation between webpages and improves user experience.

React Router

- React Router is a library that provides routing capabilities for React applications.
- `npm install react-router-dom`

Core Components of React Router

1. Browser Router: The main wrapper that enables routing in your app.

```
import { BrowserRouter } from "react-router-dom";  
<BrowserRouter>  
  <App />  
</BrowserRouter>
```

2. Routes: A container that holds all route definitions. It looks at current URL & Chooses which route to render

```
<Routes>  
  { /* routes go here */ }  
</Routes>
```

React Router

Core Components of React Router

3. Route: Defines a single path and its component

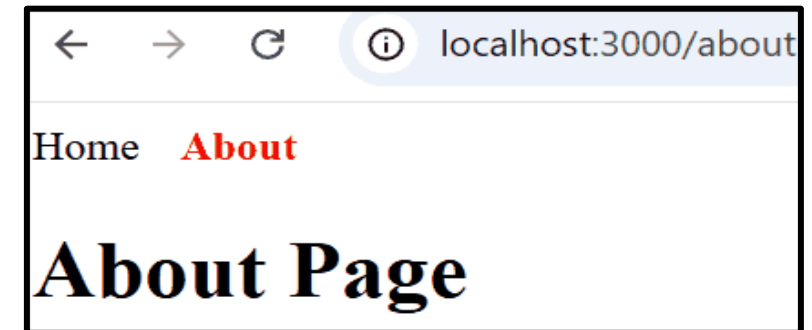
- Matches URL
- Renders corresponding component
- `<Route path="/about" element={<About />} />`
- If URL = `/about` the Show About component

4. Link: Used for navigation (instead of `<a>` tag)

- Changes URL & Prevents page reload
- `<Link to="/about">Go to About</Link>`

5. NavLink: Special version of Link

- Adds **active styling** automatically
- `import { NavLink } from "react-router-dom";`
- `<NavLink to="/about">About</NavLink>`



```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";  
import Home from "./Home";  
import About from "./About";  
import Contact from "./Contact";
```

```
function App() {  
  return (  
    <BrowserRouter>  
      <div style={{ textAlign: "center" }}>  
        <h1>My Website</h1>  
  
        <Link to="/home">Home</Link> |  
        <Link to="/about"> About</Link> |  
        <Link to="/contact"> Contact</Link>
```

```
<Routes>
```

```
{/* No default component for "/" */}
```

```
<Route path="/home" element={<Home />} />
```

```
<Route path="/about" element={<About />} />
```

```
<Route path="/contact" element={<Contact />} />
```

```
</Routes>
```

```
</div>
```

```
</BrowserRouter>
```

```
);
```

```
}
```

```
export default App;
```

```
function Home() {  
  return (  
    <div>  
      <h2>Home Page</h2>  
      <p>Welcome to the Home page of our React application.</p>  
    </div>  
  );  
}
```

```
export default Home;
```

```
function About() {  
  return (  
    <div>  
      <h2>About Page</h2>  
      <p>This page contains information about our website.</p>  
    </div>  
  );  
}
```

```
export default About;
```

```
function Contact() {  
  return (  
    <div>  
      <h2>Contact Page</h2>  
      <p>You can contact us at: example@email.com</p>  
    </div>  
  );  
}
```

```
export default Contact;
```

<Routes>

<Route path="/" element={<Home />} /> (Default Component)

<Route path="/about" element={<About />} />

<Route path="/contact" element={<Contact />} />

</Routes>